



Uzhavan Direct

Farm-to-Table, Direct. A zero-commission agricultural marketplace with a three-tier FPO logistics backbone and an in-app notification system.

Static SPA

3 Roles

3 Languages

3 Themes

FPO 3-Tier Portal

Express + PostgreSQL API

In-App Notifications

Final Project Report

Smart India Hackathon 2026 · Repository: `uzhavandirect` · Branch `main`

Report generated: 2 October 2026

This is the final consolidated report for the Uzhavan Direct build. It documents the complete system as it stands today: the vanilla-JS single-page application, the Express + PostgreSQL backend, the three-tier FPO logistics data model that is now persisted server-side, and the new in-app notification system — followed by the testing evidence, an honest error review, and the remaining limitations.

1 · Executive Summary

Uzhavan Direct is a zero-commission farm-to-table marketplace. Farmers list produce and open auctions, consumers buy and track deliveries, and a Farmer Producer Organisation (FPO) aggregates small lots into consolidated corridor shipments. The front end is a single-page app written in plain HTML/CSS/JavaScript with a deliberate localStorage demo mode; the back end is an Express API over PostgreSQL that has now become the persistent source of truth for every workflow.

This generation completed the two items that were previously local-only: the **FPO Tier-1/2/3 logistics fields** (grading, weighment, corridor, vehicle, EV agent) and the **notification ledger**. Both now live in PostgreSQL, are written through authenticated, role-checked endpoints, and are mirrored back into the existing localStorage shapes so the unchanged UI keeps rendering correctly.

3

USER ROLES

27

API ENDPOINTS

7

DATABASE TABLES

32/32

BACKEND TESTS

42/42

LIVE SMOKE CHECKS

What changed in this generation

- **NEW** `server/src/lib/notify.js` — the single writer for the notification ledger.
- **NEW** `server/src/routes/notifications.js` — authenticated list + mark-read endpoints.
- **NEW** `notifications` table with type/read/timestamp/related-record columns.
- **EXT** Order + batch routes now raise notifications inside the same transaction as the change.
- **EXT** `api.js` gained a `notifications` facade, a mirror function, and offline-safe sync.
- **EXT** The existing topbar bell is now backed by real server data — no UI redesign.

All prior work is intact: farmer, consumer and FPO API login; produce, auctions, bidding, orders and delivery stages; the FPO batch lifecycle; the multi-language, multi-theme UI; and the offline demo fallback. Nothing was rebuilt from scratch and no framework was introduced.

2 · System Architecture

The app keeps a clean three-layer split. The browser never calls the database directly; every read and write flows through the API layer, which falls back to localStorage only when the backend is unreachable.

Layer	Technology	Responsibility
-------	------------	----------------

Presentation	<code>index.html</code> + <code>style.css</code> + <code>app.js</code>	Single-page UI: 3 roles, 3 languages (Tamil/Hindi/English), 3 themes, animations, responsive layout, all views and actions.
API layer	<code>api.js</code>	The only place that talks to the backend. Typed facade, JWT handling, stage mapping, backend→local normalisers, mirror/write-through helpers, offline probe and status chip.
Backend	Express + <code>pg</code>	REST API, JWT auth, role checks, validation, transactions, PostgreSQL access.
Database	PostgreSQL (<code>pg-mem</code> in demo)	7 tables: users, otp_codes, produce, auctions, fpo_batches, orders, notifications.

Front-end ↔ back-end contract

- `api.js` loads before `app.js` (`index.html`), so the API helpers exist by the time the UI runs.
- A lightweight `/api/health` probe drives the topbar chip: ● `API` when reachable, ● `Demo` when not.
- Every write is optimistic locally, then mirrored to the server. If the server rejects, the local copy is reverted or the error is surfaced.
- Server rows are merged into the existing localStorage shapes via `apiMergeInto()` (matched on `apiId` first), so pre-existing demo records keep working.

Backward compatibility is a first-class rule. Old localStorage records have no API fields and no FPO fields; the renderers treat `fpoGrade`, `fpoWeighKg`, `fpoCorridor` and friends as optional, so seeded demo orders and brand-new server orders render side by side without special cases.

3 · Backend API

The backend lives in `server/`. It is a conventional Express app with a shared error format (`{ ok: false, error: { code, message } }`), JWT bearer auth, and role middleware.

3.1 Files

File	Purpose
<code>server/src/index.js</code>	Server entry point; loads the app and listens.
<code>server/src/app.js</code>	Express app: JSON body parsing, CORS, router registration, error handler.
<code>server/src/db.js</code>	PostgreSQL pool + <code>query()</code> + <code>withTransaction()</code> .
<code>server/src/migrate.js</code>	Applies <code>schema.sql</code> against a real database.
<code>server/src/schema.sql</code>	All tables, indexes, and the idempotent <code>ALTER</code> migration block.
<code>server/src/lib/http.js</code>	Shared vocabulary: <code>ROLES</code> , <code>STATUSES</code> , <code>AGMARK_GRADES</code> , <code>DELIVERY_STAGES</code> , <code>HttpError</code> , <code>ok/fail</code> validators.
<code>server/src/lib/notify.js</code> NEW	The only writer of the notification ledger (see §6).
<code>server/src/middleware/auth.js</code>	<code>authenticate</code> (JWT → <code>req.user</code>) and <code>requireRole()</code> .
<code>server/src/routes/auth.js</code>	Signup, login, OTP verify, <code>/me</code> .
<code>server/src/routes/produce.js</code>	Listing create/update/list/detail.
<code>server/src/routes/auctions.js</code>	Auction create/list/detail/bid.
<code>server/src/routes/orders.js</code>	Order create/list/detail, status, stage, FPO grading, EV agent.
<code>server/src/routes/fpo.js</code>	FPO inbound queue, batch create/list/patch.
<code>server/src/routes/notifications.js</code> NEW	Notification list + mark-read.
<code>server/dev-memory.js</code>	Boots the whole app on an in-memory PostgreSQL (<code>pg-mem</code>) with demo users + produce, for demos and smoke tests.
<code>server/test/api.test.js</code>	32 integration tests on the real Express app.
<code>server/scripts/smoke-live.js</code>	42 live checks against a running server.

3.2 Endpoint inventory (27 routes)

Verb	Path	Access	Purpose
POST	/api/auth/signup	public	Create an account (farmer / consumer / FPO).
POST	/api/auth/verify-otp	public	Complete the demo OTP step.
POST	/api/auth/login	public	Exchange credentials for a JWT.
GET	/api/auth/me	auth	Validate the saved token and return the current user.
POST	/api/auth/logout	auth	Client-side session close.
GET	/api/produce	public	List listings (<code>?limit</code> , filters).
GET	/api/produce/:id	public	Single listing.
POST	/api/produce	farmer	Create a listing.
PUT	/api/produce/:id	owner	Update own listing.
GET	/api/auctions	public	List auctions.
GET	/api/auctions/:id	public	Auction detail.
POST	/api/auctions	farmer	Open an auction on an owned listing.
POST	/api/auctions/:id/bid	auth	Place a bid (must beat the current highest).
POST	/api/orders	consumer	Place an order against a listing.
GET	/api/orders	auth	Role-scoped order list.
GET	/api/orders/:id	participant	Order detail (joined names + FPO fields).
PATCH	/api/orders/:id	farmer / fpo	Status change (pending / completed / refunded).
PATCH	/api/orders/:id/stage	farmer / fpo	Advance the 7-step delivery pipeline.
PATCH	/api/orders/:id/grading	fpo	Tier-1 AGMARK grade + weighment + timestamp.
PATCH	/api/orders/:id/agent	fpo	Tier-3 EV agent assignment + <code>outForDelivery</code> .
GET	/api/fpo/inbound	fpo	Tier-1 lots awaiting grading (<code>?include_graded=true</code>).
GET	/api/fpo/batches	fpo	This FPO's corridor shipments.
GET	/api/fpo/batches/:id	fpo	Batch detail with member orders.
POST	/api/fpo/batches	fpo	Assemble a corridor batch from graded lots.
PATCH	/api/fpo/batches/:id	fpo	assembling → dispatched → received.
GET	/api/notifications	auth	NEW Current user's notifications + unread/total.
PATCH	/api/notifications/:id/read	owner	NEW Mark one notification read (404 for foreign ids).
GET	/api/health	public	Liveness probe used by the status chip.

3.3 Conventions followed

- **Auth:** `Authorization: Bearer <jwt>` , decoded by `authenticate` into `req.user = { id, role, name }` .
- **Roles:** `requireRole('fpo')` etc. Unauthorised roles get `403 FORBIDDEN` , missing tokens get `401 UNAUTHENTICATED` .
- **Validation:** every id passes `asId()` ; grades are checked against `AGMARK_GRADES` ; weights must be a finite number `> 0` ; malformed input yields `400 VALIDATION_ERROR` .
- **Errors:** a single shape — `{ ok:false, error:{ code, message } }` — with stable codes such as `NOT_FOUND` , `FORBIDDEN` , `STAGE_REGRESSION` , `BATCH_NOT_RECEIVED` .
- **Transactions:** `withTransaction()` wraps multi-statement writes (grading + notification, dispatch + fan-out, agent + stage) so a failure rolls back atomically.

4 · Database Schema

Seven tables, all created with `CREATE TABLE IF NOT EXISTS` so the schema is safe to re-apply. This generation added the `notifications` table and the FPO tier columns on `orders`. The FPO columns are also applied through an idempotent `ALTER ... ADD COLUMN IF NOT EXISTS` block so an older database is upgraded in place without touching existing rows.

4.1 Tables

Table	Key columns
users	id, role (farmer/consumer/fpo), name, phone, password_hash, village, cluster, hub.
otp_codes	Signup OTP ledger (phone, code, expires_at, used).
produce	id, farmer_id, name, category, quantity, price, unit, image_url.
auctions	id, produce_id, farmer_id, base_rate, highest_bid, highest_bidder, ends_at, sold.
orders	Consumer/farmer/fpo ids, quantity, status, delivery_stage, plus the FPO tier columns below.
fpo_batches	id, fpo_id, source_cluster, destination_hub, consolidated_weight_kg, vehicle_type, transit_status.
notifications	See §6.2.

4.2 FPO tier columns added to orders

Column	Type	Meaning
fpo_grade	TEXT, CHECK ∈ A/B/C	Tier-1 AGMARK grade. NULL for ungraded lots.
fpo_weigh_kg	NUMERIC, NULL-safe ≥ 0	Tier-1 digital weighment. The route enforces > 0 (min: 0.01); the column stays NULL-safe for legacy rows.
fpo_graded_at	TIMESTAMP TZ	Set with now() inside the grading statement — server time, never client time.
fpo_batch_id	INTEGER FK → fpo_batches	Which corridor shipment carries this lot.
fpo_corridor	TEXT	Tier-2 corridor, e.g. Nilgiris cluster → Thanjavur hub.
fpo_vehicle	TEXT	Tier-2 vehicle type (truck / mini-truck / tempo).
fpo_ev_agent	TEXT	Tier-3 EV doorstep agent, e.g. Kumar · EV-01.
fpo_ev_assigned_at	TIMESTAMP TZ	When the EV agent was assigned (server time).

4.3 Migration safety

- **Non-destructive:** no `DROP`, no `DELETE`, no column type change. Existing orders and batches keep their data.
- **Idempotent:** the `ALTER ... IF NOT EXISTS` block is a no-op on a fresh database and self-heals an old one.
- **pg-mem-compatible:** every CHECK is written NULL-safely (`fpo_grade IS NULL OR fpo_grade IN (...)`) so the in-memory engine used by the demo and tests accepts it, and `COUNT(*) FILTER` is avoided in queries.
- **Referential integrity:** notification `order_id` / `batch_id` use `ON DELETE SET NULL` so removing an order never destroys alert history; `user_id` uses `ON DELETE CASCADE`.
- **Indexes:** `idx_orders_fpo_batch`, `idx_notifications_user`, and `idx_notifications_unread` keep the FPO and bell queries cheap.

5 · FPO Three-Tier Logistics Workflow

The FPO portal aggregates small farm lots into consolidated corridor shipments. All three tiers are now backed by the database; the local values are a mirror that the unchanged UI renders.



5.1 Tier-1 — AGMARK grading & digital weighment

- **Queue:** `GET /api/fpo/inbound` returns lots that have reached the collection point and are unclaimed or owned by this FPO (`?include_graded=true` also returns already-graded ones).

- **Action:** `PATCH /api/orders/:id/grading` with `{ fpo_grade, fpo_weigh_kg }`.
- **Behaviour:** writes grade, weight, `fpo_graded_at = now()`, and claims the lot (`fpo_id`) in one transaction, then notifies the farmer and consumer.
- **Guards:** FPO-only; a lot already claimed by another FPO → `403`; a lot that has not reached the collection point → `409 STAGE_NOT_ALLOWED`; bad grade or non-positive weight → `400`.

5.2 Tier-2 — corridor, vehicle & consolidation

- **Assemble:** `POST /api/fpo/batches` with source cluster, destination hub, consolidated weight, vehicle type, and the list of member order ids.
- **Stamping:** every attached order receives `fpo_batch_id`, `fpo_corridor` and `fpo_vehicle` via `COALESCE` (an existing value is never overwritten), so the corridor and vehicle the FPO chose become backend truth for each lot.
- **Lifecycle:** `PATCH /api/fpo/batches/:id` moves `assembling` → `dispatched` → `received`.
- **Notifications:** a milestone alert fires only when `transit_status` actually enters `dispatched` or `received` — repeated saves of the same state do not spam the consumer.

5.3 Tier-3 — EV doorstep agent

- **Action:** `PATCH /api/orders/:id/agent` with `{ agent: 'Kumar · EV-01' }`, which also advances the order to `outForDelivery` in the same statement.
- **Atomically consistent:** assignment and stage can never contradict each other because they are one update.
- **Guards:** if the order travelled on a corridor batch, that batch must have been received first → `409 BATCH_NOT_RECEIVED`; an already-delivered order → `409 STAGE_REGRESSION`.

5.4 Field mapping — backend → existing front-end shapes

Backend column	Front-end field	Where it renders
<code>fpo_grade</code>	<code>fpoGrade</code>	FPO lot card, consumer tracking
<code>fpo_weigh_kg</code>	<code>fpoWeighKg</code>	FPO lot card, batch weight total
<code>fpo_graded_at</code>	<code>fpoGradedAt</code>	FPO lot card timestamp
<code>fpo_corridor</code>	<code>fpoCorridor</code>	FPO batch card, tracking banner
<code>fpo_vehicle</code>	<code>fpoVehicle</code>	FPO batch card
<code>fpo_ev_agent</code>	<code>fpoEvAgent</code>	Tier-3 assignment panel, consumer tracking
<code>fpo_ev_assigned_at</code>	<code>fpoEvAssignedAt</code>	Tier-3 assignment panel timestamp

6 · In-App Notification System

The existing topbar bell already existed in the UI. This generation made it real: notifications are now stored server-side, retrieved per user, and marked read on the backend. No push, email, SMS, Firebase or external service was added — it is a purely in-app ledger feeding the existing dropdown.

6.1 Where it lives

Piece	Role
<code>server/src/lib/notify.js</code>	The only module that inserts into <code>notifications</code> . Exposes <code>createNotification</code> , <code>notifyOrderParticipants</code> , <code>notifyManyOrders</code> , <code>selectBatchOrders</code> , and the shared <code>STAGE_LABELS</code> .
<code>server/src/routes/notifications.js</code>	Authenticated <code>GET /api/notifications</code> and <code>PATCH /api/notifications/:id/read</code> .
<code>api.js</code> (front end)	<code>notifications</code> facade, <code>notifFromApi()</code> , <code>syncNotifications()</code> , <code>markNotificationReadOnBackend()</code> .
<code>app.js</code> (front end)	Existing <code>refreshNotifBadge()</code> / <code>renderNotifPanel()</code> / <code>markNotifRead()</code> , now backend-aware.

6.2 Table `notifications`

Column	Type	Notes
<code>id</code>	<code>SERIAL PRIMARY KEY</code>	—

<code>user_id</code>	<code>INTEGER NOT NULL</code> FK users	Recipient. <code>ON DELETE CASCADE</code> .
<code>type</code>	<code>TEXT</code> , CHECK ∈ order/grade/fpo/system	Grouping shown in the dropdown.
<code>message</code>	<code>TEXT NOT NULL</code>	Human-readable text (capped at 300 chars on write).
<code>order_id</code>	<code>INTEGER</code> FK orders	<code>ON DELETE SET NULL</code> .
<code>batch_id</code>	<code>INTEGER</code> FK fpo_batches	<code>ON DELETE SET NULL</code> .
<code>is_read</code>	<code>BOOLEAN NOT NULL DEFAULT FALSE</code>	Unread drives the badge.
<code>created_at</code>	<code>TIMESTAMPZ NOT NULL DEFAULT now()</code>	Server time, newest-first ordering.

6.3 Events that raise a notification

Workflow event	Type	Recipients
Order placed / status changed	<code>order</code>	Farmer and FPO (not the acting user).
Delivery stage advanced	<code>order</code>	Farmer and FPO as the parcel moves; wording from <code>STAGE_LABELS</code> .
Tier-1 grading completed	<code>grade</code>	Consumer and farmer: "<produce> graded A · 18.5 kg at the farm gate".
Tier-3 EV agent assigned	<code>fpo</code>	Consumer: "Kumar · EV-01 picked up order #1 — out for delivery".
FPO batch dispatched	<code>fpo</code>	Every consumer/farmer on the shipment's orders.
FPO batch received at hub	<code>fpo</code>	Same fan-out, once per transition.

Rules that keep the bell trustworthy

- **Never notify the actor.** `notifyOrderParticipants()` filters out `req.user.id`, so an FPO is never told about its own grading.
- **Transactional.** The notification is written inside the same `withTransaction()` as the change; a rolled-back action leaves no stray alert.
- **No transition spam.** Batch milestone alerts fire only on a real `dispatched` / `received` transition.

6.4 Front-end behaviour

- **On login and on boot:** `apiSyncAll()` calls `syncNotifications()`, which replaces the server-owned entries (`apiId`) in `ud_users[user].notifications` and keeps local-only ones, then re-sorts newest-first.
- **Badge:** the existing `refreshNotifBadge()` shows the unread count; the dropdown lists recent alerts with relative dates.
- **Mark as read:** local flag flips instantly for responsive UI; `markNotificationReadOnBackend()` confirms with the server and *reverts* the local flag if the server rejects, so a future sync cannot silently flip the badge back.
- **Offline:** the probe fails fast, `syncNotifications()` returns quietly, and the bell keeps showing whatever it already held — no error, no cleared data.
- **No duplicates:** `notifyOrderConsumer()` returns early for backend-owned orders (those with an `apiId` while online), because the server already created that alert. Seeded demo orders and offline actions still get a local copy.

7 · Front-End API Layer (`api.js`)

`api.js` is the single API layer — no second layer was created. It owns the token, the probe, the stage mapping, all backend→local normalisers, and one small write-through helper per UI action.

7.1 Facade

Group	Methods
<code>api.auth</code>	<code>login</code> , <code>signup</code> , <code>verifyOtp</code> , <code>me</code> , <code>logout</code>
<code>api.produce</code>	<code>list</code> , <code>get</code> , <code>create</code> , <code>update</code>
<code>api.auctions</code>	<code>list</code> , <code>get</code> , <code>create</code> , <code>bid</code>
<code>api.orders</code>	<code>list</code> , <code>get</code> , <code>create</code> , <code>setStatus</code> , <code>setStage</code> , <code>setGrading</code> , <code>assignAgent</code>
<code>api.fpo</code>	<code>inbound</code> , <code>listBatches</code> , <code>getBatch</code> , <code>createBatch</code> , <code>updateBatch</code>

api.notifications	list(params), markRead(id)
NEW	

7.2 Key helpers

- apiRequest() — one fetch wrapper: base URL, JSON, bearer token, timeout, standard error normalisation, and a single place that reacts to 401.
- apiProbe() — cached /api/health check that drives the status chip and gates every write-through helper.
- toApiStage() / fromApiStage() — maps the front end's 8 stages onto the backend's 7 (the extra atCustomerCity maps to atFpo) so the existing tracking UI is untouched.
- apiPasswordFor() — pads the short demo passwords to the API's 6-character minimum (123 → ud-123-demo), identically on the server seed.
- apiMergeInto() — merges server rows into a localStorage key, matching on apiId first and falling back to natural keys, so re-syncing never duplicates.
- Write-through helpers — syncOrderStageToApi, syncOrderStatusToApi, syncFpoGradeToApi, syncEvAgentToApi, syncNewBatchToBackend, syncBatchStatusToApi, plus produce/auction/cart equivalents.

7.3 Functions connected to the new workflows

Each existing UI action gained exactly one write-through call; the views themselves were not rewritten.

Front-end action	Backend call
confirmFpoGrade(orderId)	PATCH /api/orders/:id/grading → grade + weight + timestamp + claim
createFpoBatch()	POST /api/fpo/batches → corridor + vehicle + member orders
dispatchFpoBatch() / receiveFpoBatch()	PATCH /api/fpo/batches/:id → dispatched / received
assignFpoEvAgent()	PATCH /api/orders/:id/agent → EV agent + outForDelivery
setOrderStage() / status change	PATCH /api/orders/:id/stage / /:id
markNotifRead(id)	PATCH /api/notifications/:id/read
Login / boot	GET /api/notifications via syncNotifications()

8 · Security & Validation

Validation happens on the server, never only in the browser. The table below lists the guarantees that are covered by tests and smoke checks.

Threat / mistake	Response	Where enforced
Invalid AGMARK grade	400 VALIDATION_ERROR	oneOf(..., AGMARK_GRADES) in the grading route + DB CHECK
Zero / negative weight	400 VALIDATION_ERROR	asNumber(..., { min: 0.01 })
Farmer or consumer trying to grade	403 FORBIDDEN	requireRole('fpo') on grading, agent and batch routes
Non-FPO assigning an EV agent	403 FORBIDDEN	requireRole('fpo') on /:id/agent
A different FPO grading a claimed lot	403 FORBIDDEN	Explicit fpo_id owner check in the grading route
Invalid / missing order or batch id	400 or 404 NOT_FOUND	asId() + existence lookups
Agent assigned before the batch is received	409 BATCH_NOT_RECEIVED	Tier-3 guard
Regressing a delivered order	409 STAGE_REGRESSION	Stage pipeline guard
Reading another user's notifications	Scoped query; foreign id → 404	WHERE user_id = req.user.id + owner check in the read route
Missing / expired token	401 UNAUTHENTICATED	authenticate middleware; client clears the session on 401
Malformed JSON body	400 via the shared error handler	requireFields() + body parser

9 · Front-End UI & Experience

The interface was deliberately left alone. It remains a single static page — no build step, no framework, just HTML/CSS/JS.

9.1 Roles

Role	Core capabilities
Farmer	List produce with photos, open and manage auctions, view orders, advance delivery stages, ratings, donations received, AI market advisor.
Consumer	Browse marketplace, cart and checkout, track orders across the delivery pipeline, follow farmers, donate, rate, pick a delivery slot.
FPO	The three-tier portal: Tier-1 inbound grading queue, Tier-2 corridor batch assembly and dispatch, Tier-3 EV agent assignment and hub receipt.

9.2 Systems preserved

- Three languages — Tamil, Hindi, English (`langSelect`).
- Three themes plus a light/dark colour mode.
- Responsive layout with a collapsible sidebar.
- Animations and toasts as before.
- Leaflet map for delivery tracking.
- Cart drawer and checkout flow.
- Auction bidding UI with live countdown.
- Farmer ratings and donation ledger.
- Topbar API status chip (`API` / `Demo`).
- Notification bell with unread badge and dropdown.
- AI market advisor chat.
- Offline localStorage demo mode.

Design constraint honoured: this generation added no new view, no new route, and no visual redesign. The only UI-visible addition is that the existing bell now shows real server notifications, and the existing FPO cards now survive a page reload because their values come from the database.

10 · Testing & Verification

10.1 Backend test suite — 32/32 passing

`cd server && npm test` runs Node's built-in test runner against the real Express app on an in-memory PostgreSQL. No pre-existing test was modified or deleted.

Area	What it verifies
Auth	Signup, OTP, login, <code>/me</code> , bad credentials, role mapping.
Produce	List, create, ownership on update, validation.
Auctions	Create, list, detail, bid below highest rejected, bid accepted.
Orders	Create, role-scoped list, detail, status, stage pipeline + regressions.
FPO Tier-1	FPO grades an order; valid weight recorded; invalid weight rejected; consumer cannot grade (403) ; grade/weight/timestamp survive a fresh GET.
FPO Tier-2/3	Corridor + vehicle persisted; EV agent + timestamp persisted; batch dispatch/receive; direct dispatch when no batch.
Notifications <code>NEW</code>	Workflow events notify the right people; list, mark read, and cross-user isolation.
Unauthenticated writes	Write endpoints reject missing tokens.

10.2 Live smoke test — 42/42 checks passing

`cd server && npm run smoke` exercises a running server end-to-end. New checks added this generation cover the FPO operations and the whole notification surface.

Sample of the notification checks

- `GET /api/notifications (consumer) - 7 unread / 7 total`
- `workflow events produced notifications - fpo, grade, order`
- `at least one unread notification - Kumar · EV-01 picked up order #1 - out for delivery.`
- `PATCH /api/notifications/:id/read → then unread_only no longer lists it`
- `another user's notification → 404 NOT_FOUND`
- `FPO is never notified about its own actions - 0 grade alerts`
- `notifications require auth → 401 UNAUTHENTICATED`

10.3 Browser verification

- Consumer login → chip `API`; bell badge showed the unread count; the dropdown rendered the mirrored alerts with correct messages and dates.
- Clicking **Mark as read** flipped the badge and the backend confirmed `is_read = true` with the unread count decreasing.
- **Offline degradation:** with the API state flipped offline in-page, the chip showed `Demo`, the bell kept its existing entries and badge, and marking a notification read still worked locally — no errors, no data loss.
- **Offline mark-read:** verified the local flag flips and the server is left untouched when the backend is unreachable.
- **Online mark-read:** verified the server row is updated and the unread count falls accordingly.

Resolved anomaly (was listed as unresolved). An earlier session observed that reloading the page while offline appeared to empty the bell. Investigation this generation showed the cause was the *preview browser's incognito context clearing* `localStorage` on every reload — a test-harness artifact, not an application bug. A plain key written and read back across a reload vanished the same way. The app's own offline path was then verified directly and behaves correctly: the bell retains its entries, degrades to the local copy, and never throws.

11 · Error Review

A deliberate review pass was run over the whole generation before producing this report.

Check	Result	Notes
Syntax check on every source file	Pass	<code>node --check</code> on <code>api.js</code> , <code>app.js</code> and all <code>server/**</code> files produced no errors.
Backend test suite	Pass	32/32.
Live smoke test	Pass	42/42.
Duplicate notifications	Pass	The server writes for backend-owned orders; <code>notifyOrderConsumer()</code> early-returns for those, so the bell shows one row per event.
Notification on the actor	Pass	<code>notifyOrderParticipants()</code> excludes <code>req.user.id</code> ; smoke asserts an FPO receives 0 grade alerts for its own grading.
Cross-user notification access	Pass	Queries scoped to <code>req.user.id</code> ; foreign id returns <code>404</code> .
Batch milestone spam	Pass	Alerts fire only on a real <code>dispatched / received</code> transition.
Transaction integrity	Pass	Grading, dispatch/receive and agent assignment each write their notification inside the same transaction.
NULL safety on old records	Pass	All FPO fields are optional in the renderers; legacy orders render with empty/null values.
Line-ending safety	Pass	<code>app.js</code> / <code>index.html</code> / <code>style.css</code> are CRLF and were edited with CRLF-exact replacements; <code>api.js</code> and <code>server/**</code> are LF.
Stage mapping edge cases	Pass	The 8↔7 stage mapping (<code>atCustomerCity</code> → <code>atFpo</code>) is handled explicitly; expected stage disagreements are logged, not surfaced as errors.
Outstanding TODO markers	None relevant	The only <code>TODO</code> is a pre-existing, deliberately unimplemented SMS-provider note in <code>auth.js</code> — out of scope for this step.

No logical or runtime errors were found in the completed work. The one previously open item (the offline bell) was root-caused to the test harness and is documented in §10.3.

12 · Development Report (10 items)

1 · Files created

- `server/src/lib/notify.js` — shared notification writer.
- `server/src/routes/notifications.js` — list + mark-read endpoints.
- `server/dev-memory.js`, `server/scripts/smoke-live.js` — in-memory demo server and live smoke suite (from the integration step).

2 · Files modified

- `server/src/schema.sql` — `notifications` table + FPO tier columns and migration block.
- `server/src/app.js` — registers the notifications router.
- `server/src/routes/orders.js` — grading/agent/stage/status now transactional with notifications.
- `server/src/routes/fpo.js` — batch dispatch/receive milestone notifications.
- `server/test/api.test.js` — new grading + notification coverage (32 tests).
- `server/scripts/smoke-live.js` — notification smoke checks (42 checks).
- `api.js` — notifications facade, mirror + sync, mark-read, and write-through calls.
- `app.js` — `markNotifRead()` backend call and duplicate-alert guard.

3 · Database changes

- New table `notifications` (id, user_id, type, message, order_id, batch_id, is_read, created_at) + two indexes.
- FPO tier columns on `orders`: `fpo_grade`, `fpo_weigh_kg`, `fpo_graded_at`, `fpo_batch_id`, `fpo_corridor`, `fpo_vehicle`, `fpo_ev_agent`, `fpo_ev_assigned_at`.
- Idempotent, non-destructive `ALTER ... ADD COLUMN IF NOT EXISTS` migration; NULL-safe CHECKs.

4 · New API endpoints

- `GET /api/notifications` — current user's notifications with unread/total counts.
- `PATCH /api/notifications/:id/read` — mark one read (owner only).
- `PATCH /api/orders/:id/grading`, `PATCH /api/orders/:id/agent`, `GET /api/fpo/inbound` — FPO tier operations (grading, EV agent, inbound queue).

5 · Front-end functions connected

- `confirmFpoGrade()`, `createFpoBatch()`, `dispatchFpoBatch()`, `receiveFpoBatch()`, `assignFpoEvAgent()` — each now writes through to the backend.
- `setOrderStage()` / status changes — mirrored to the API.
- `markNotifRead()` — calls `markNotificationReadOnBackend()`.
- `apiSyncAll()` — calls `syncNotifications()` on login and boot.

6 · Notification behaviour

Order status/stage changes, Tier-1 grading completion, Tier-3 EV assignment, and FPO batch dispatch/receive all create server-side notifications for the other participants (never the actor). The bell shows the unread count, lists recent alerts, and marks them read on the backend; it fails gracefully offline by keeping its local copy.

7 · Tests added

- FPO can grade; valid weight recorded; **invalid weight rejected; consumer cannot grade (403)**; grade/weight/timestamp survive a fresh GET.
- Notifications: workflow events notify the right people; list, mark read, and cross-user isolation.

8 · Existing tests result

All green — 32/32 pass. No existing test was modified or deleted.

9 · New smoke-test result

All green — 42/42 live checks pass, including the FPO operations and the full notification surface.

10 · Remaining limitation

The notification system is intentionally in-app only: no push, email, SMS, or realtime delivery, so a badge refreshes on login/boot or after a local action rather than the instant another user acts. Multi-device realtime sync, offline write queueing, refresh tokens, and httpOnly-cookie sessions remain future work, as agreed.